

Your first agent

Welcome to *Your first agent*  tutorial! Here you will learn from the very basics of the framework how to create agents with pure Python. Once you finish this 5 minutes guide, you will be ready to build your own agents. Take advantage of our documentation to deepen into anything you need for your agents.

The greetings agent

The agents we are going to create has a really simple structure. The user is intended to greet the agent and answer a simple *How are you?* question. This workflow is repeated *ad infinitum*.

You can also see the full agent code [here](#)

The following code should be saved in a Python file named *greetings_agent.py*.

Import the dependencies

Before starting coding the agent itself, we need to import the necessary dependencies (make sure you have installed the core and torch and extras requirements from the requirements folder as they will be needed for the agent to run).

```
from baf.core.agent import Agent
from baf.core.session import Session
```

Optionally, you can configure the logging system to display log messages at specific levels (INFO, WARNING...). By default, the logger is at INFO level, which displays all the agent actions through the terminal. This level can help you understand the agent behaviour.

```
import logging
from baf.exceptions.logger import logger
logger.setLevel(logging.WARNING)
```

Create the agent

Let's start creating your agent. We only need to specify its name.

```
agent = Agent('greetings_agent')
```

Note

Some agent components may need you to specify some properties. Some may be compulsory (e.g. an API key), others can be optional and others may have default values so you don't necessarily need to specify them.

For this agent we don't need to set any property, but you can see the [configuration properties](#) to learn more.

Define the platform your agents will use.

```
websocket_platform = agent.use_websocket_platform(use_ui=True)
```

The agent platform will allow you to communicate with your agents using a bidirectional channel, so you can send and receive messages.

States

Now, we are going to create the agent states you can see in the previous figure.

Warning

Every agent has 1 (and only 1) initial state! See [States](#) to learn more.

```
initial_state = agent.new_state('initial_state', initial=True)
hello_state = agent.new_state('hello_state')
good_state = agent.new_state('good_state')
bad_state = agent.new_state('bad_state')
```

Intents

The next step is to define all the intents for the agent. *Intent* refers to the specific purpose or goal that a user has when interacting with the agents.

An intent is composed by a name, a set of training sentences and optionally a set of parameters (not necessary now).

The idea here is to give representative examples of each intent so the agent can understand the users messages and identify their intents.

Usually, the more examples you provide (what we call training data), the better predictions the agent will make on the users inputs. But note that data quality matters!

```
hello_intent = agent.new_intent('hello_intent', [
    'hello',
    'hi',
])

good_intent = agent.new_intent('good_intent', [
    'good',
    'fine',
])

bad_intent = agent.new_intent('bad_intent', [
    'bad',
    'awful',
])
```

State bodies and transitions

Once we have all the agent components, let's define the state bodies and the transitions.

The body of a state is a python function where you can do anything you want.

It will be run whenever the agent transitions to its state.

It receives the user session as a parameter to read/write user-specific information.

Note

The agent can send messages to the user through the user session (`session.reply("message")`) or through the platform (`websocket_platform.reply(session, "message")`). There are other kinds of replies which can be platform-specific (e.g. sending a picture, reacting to a user message...)

initial_state

This state has a transition to *hello_state* that is triggered when the agent receives the *hello_intent*. The state body is not defined since this state does nothing.

```
initial_state.when_intent_matched(hello_intent).go_to(hello_state)
```

hello_state

This state can transition to *good_state* or *bad_state* depending on the user response.

```
def hello_body(session: Session):
    session.reply('Hi! How are you?')

hello_state.set_body(hello_body)
hello_state.when_intent_matched(good_intent).go_to(good_state)
hello_state.when_intent_matched(bad_intent).go_to(bad_state)
```

good_state

Here the agent replies according with the last user intent (*good_intent*).

This state has an automatic transition to *initial_state*.

```
def good_body(session: Session):
    session.reply('I am glad to hear that!')

good_state.set_body(good_body)
good_state.go_to(initial_state)
```

bad_state

Here the agent replies according with the last user intent (*bad_intent*).

This state has an automatic transition to *initial_state*.

```
def bad_body(session: Session):
    session.reply('I am sorry to hear that...')

bad_state.set_body(bad_body)
bad_state.go_to(initial_state)
```

Run the agent

Everything is ready to run the agent!

```
if __name__ == '__main__':
    agent.run()
```

Finally, open a terminal and run the agent script:

```
python greetings_agent.py
```

Once the agent is trained, a web browser tab with the agents interface will pop up and you will be able to start chatting!

If you encounter the following error:

```
ModuleNotFoundError: No module named 'besser'
```

You need to add the following code lines at the beginning of the agent script, to add your working directory to the Python path:

```
import sys
sys.path.append("/Path/to/directory/agent-framework") # Replace with your
directory path
```